

Recall

What is a PDA?

Formally, a Pushdown Automaton (PDA) M is defined as a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ where:

- ▶ Q is a finite set called the states.
- ▶ Σ is the set of input alphabets.
- ▶ Γ is the set of Stack alphabets.
- ▶ $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$ is the transition function (where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ and $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$).
- ▶ $q_0 \in Q$ is the start state.
- ▶ $F \subseteq Q$ is the set of accepting (or final) states.

Recall

Grammars

Formally, a Grammar G is a 4-tuple (V, Σ, P, S) such that:

- ▶ V is the set of Variables
- ▶ Σ is the set of Terminals
- ▶ P is the set of production Rules
 $[(V \cup \Sigma)^* V (V \cup \Sigma)^* \rightarrow (V \cup \Sigma)^*]$
- ▶ S is the Start Variable [The variable in the LHS of the first rule is generally the start variable]

Context free Grammars

If the rules of the underlying grammar G are of the form

$$V \rightarrow (V \cup \Sigma)^*$$

then such a grammar is called Context-Free.

Any language generated by a context-free grammar is called a context-free language.

Equivalence of PDAs and CFGs

- ▶ Pushdown automata and Context Free Grammars have the same computational power.
- ▶ One can establish this by showing that:
 - ▶ For every CFG an equivalent PDA exists, and
 - ▶ For every PDA an equivalent CFG exists.
- ▶ What does equivalent mean?
 - ▶ A CFG and its corresponding PDA are said to be equivalent if:
 - ▶ For every string x that can be derived by the CFG, the equivalent PDA on input x ends in an accepting state.
 - ▶ And for every string x for which the PDA halts in an accepting state (or empty stack depending on implementation), the string x can be derived by the CFG.

PDA $\rightarrow$ CFG

- ▶ So now we are given a PDA P .
- ▶ We need to show that for any given PDA P , we can create a Context Free Grammar G that is equivalent to P .
- ▶ Proof construction from Sipser.

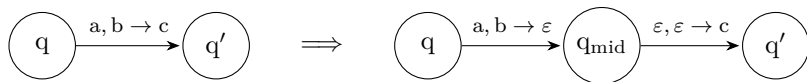
Step 1: Simplifying the PDA

Before constructing the grammar G , we modify P to satisfy three simplifying conditions:

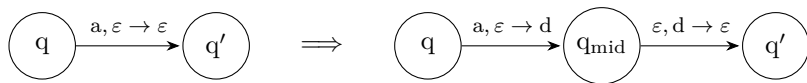
1. Single accept state: P has a single accept state, q_{accept} .
2. Empty stack on accept: P empties its stack before accepting.
3. Strict stack operations: Each transition either pushes a symbol onto the stack or pops a symbol off the stack, but does not do both at the same time (and does not do neither).

How to Enforce Condition 3 (Strict Push or Pop)

Transformation for push-and-pop transitions ($a, b \rightarrow c$):



Transformation for no-op transitions ($a, \epsilon \rightarrow \epsilon$):



Every transition now strictly pushes or strictly pops exactly one symbol.

Step 2: Grammar Construction

Variables of G :

- ▶ For every pair of states $p, q \in Q$, create a variable A_{pq} (n^2 variables).
- ▶ Meaning: A_{pq} generates all strings taking P from state p with empty stack to state q with empty stack.
- ▶ Start variable: $A_{q_0, q_{\text{accept}}}$.

Rules of G :

1. For each $p, q, r, s \in Q$, $u \in \Gamma$, and $a, b \in \Sigma_\epsilon$:

If $(r, u) \in \delta(p, a, \epsilon)$ and $(q, \epsilon) \in \delta(s, b, u)$, put $A_{pq} \rightarrow aA_{rs}b$

2. For each $p, q, r \in Q$, put $A_{pq} \rightarrow A_{pr}A_{rq}$.
3. For each $p \in Q$, put $A_{pp} \rightarrow \epsilon$.

Intuition: Stack Height vs Input String

You may gain some insight for this construction from the following figures.

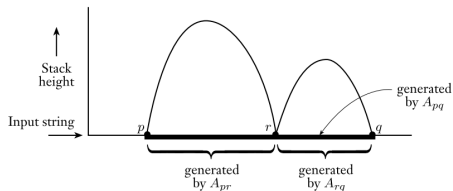
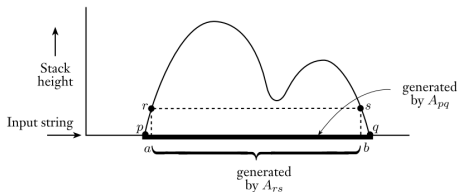


FIGURE 2.28

PDA computation corresponding to the rule $A_{pq} \rightarrow A_{pr}A_{rq}$



Correctness: The If-and-Only-If Statement

We prove that the grammar G is correct by demonstrating that for all $p, q \in Q$:

A_{pq} generates $x \iff x$ brings P from p to q with empty stack

Why is this general statement useful?

- ▶ The start variable of G is $A_{q_0, q_{\text{accept}}}$.
- ▶ Setting $p = q_0$ and $q = q_{\text{accept}}$ gives:

$A_{q_0, q_{\text{accept}}} \xRightarrow{*} x \iff x$ brings $q_0 \rightarrow q_{\text{accept}}$ with empty stack

- ▶ Since P was simplified to accept if and only if it reaches q_{accept} with an empty stack, this directly proves $L(G) = L(P)$!

Proof Strategy (Two Directions):

- ▶ Claim 1 (Only If): $A_{pq} \xRightarrow{*} x \implies x$ takes $p \rightarrow q$ with empty stack.
- ▶ Claim 2 (If): x takes $p \rightarrow q$ with empty stack $\implies A_{pq} \xRightarrow{*} x$.

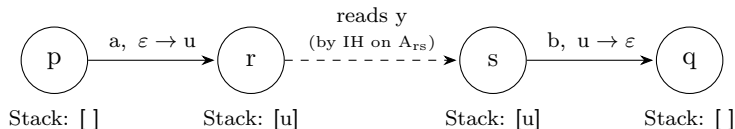
Claim 1 (Only If Direction): Setup & Basis

Claim 1: If A_{pq} generates x , then x brings P from p with empty stack to q with empty stack.

- ▶ Proof Technique: Induction on derivation length of x from A_{pq} .
- ▶ Basis (Derivation has 1 step):
 - ▶ Must use a rule with no variables on RHS: $A_{pp} \rightarrow \varepsilon$.
 - ▶ Thus $p = q$ and $x = \varepsilon$.
 - ▶ The string ε takes P from p with empty stack to p with empty stack.
 - ▶ Hence, basis holds.
- ▶ Induction Hypothesis:
 - ▶ Assume true for all derivations of length $\leq k$ ($k \geq 1$).
 - ▶ Prove true for derivations of length $k + 1$.

Claim 1: Induction Step (Case 1: $A_{pq} \Rightarrow aA_{rs}b$)

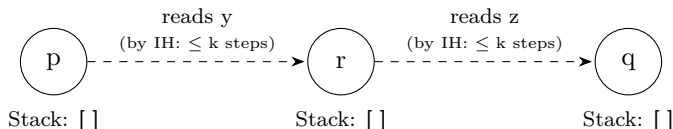
Suppose $A_{pq} \xRightarrow{*} x$ in $k + 1$ steps, and first step is $A_{pq} \Rightarrow aA_{rs}b$ ($x = ayb$).



- ▶ $A_{rs} \xRightarrow{*} y$ in k steps $\implies$ by IH, y brings r to s with empty stack.
- ▶ Rule $A_{pq} \rightarrow aA_{rs}b \implies (r, u) \in \delta(p, a, \varepsilon)$ and $(q, \varepsilon) \in \delta(s, b, u)$.
- ▶ Since y executes with empty stack, it leaves bottom symbol u untouched.
- ▶ Reading b at state s pops u and reaches state q with empty stack!

Claim 1: Induction Step (Case 2: $A_{pq} \Rightarrow A_{pr}A_{rq}$)

Suppose $A_{pq} \xRightarrow{*} x$ in $k + 1$ steps, and first step is $A_{pq} \Rightarrow A_{pr}A_{rq}$ ($x = yz$).



- ▶ $A_{pr} \xRightarrow{*} y$ and $A_{rq} \xRightarrow{*} z$ each take $\leq k$ steps.
- ▶ By Induction Hypothesis:
 - ▶ y brings P from p (empty stack) to r (empty stack).
 - ▶ z brings P from r (empty stack) to q (empty stack).
- ▶ Sequential composition: $x = yz$ takes P from $p \rightarrow q$ with empty stack.

Claim 2 (If Direction): Setup & Basis

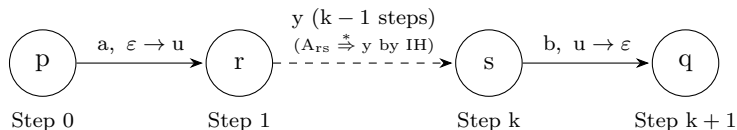
Claim 2: If x brings P from p with empty stack to q with empty stack, then A_{pq} generates x .

- ▶ Proof Technique: Induction on number of computation steps of P on x .
- ▶ Basis (Computation has 0 steps):
 - ▶ 0-step computation reads no characters, so $x = \varepsilon$.
 - ▶ Machine starts and ends in same state: $p = q$.
 - ▶ By construction, G contains the rule $A_{pp} \rightarrow \varepsilon$.
 - ▶ Thus $A_{pp} \xRightarrow{*} \varepsilon$, and basis holds.
- ▶ Induction Hypothesis:
 - ▶ Assume true for computations of length $\leq k$ ($k \geq 0$).
 - ▶ Prove true for computations of length $k + 1$.

Claim 2: Induction Step (Case 1: Stack empty only at ends)

Suppose P executes $k + 1$ steps on input x ($p \rightarrow q$ with empty stacks).

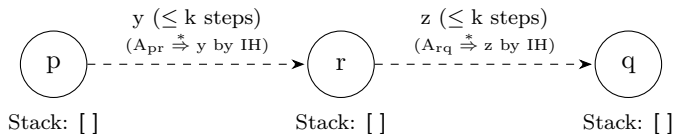
Case 1: Stack is empty only at the beginning and end ($x = ayb$).



- First move pushes $u \implies (r, u) \in \delta(p, a, \epsilon)$.
- Last move pops $u \implies (q, \epsilon) \in \delta(s, b, u)$.
- Inner computation on y takes $(k + 1) - 2 = k - 1$ steps $\implies A_{rs} \xRightarrow{*} y$ (by IH).
- Grammar derivation: $A_{pq} \Rightarrow aA_{rs}b \xRightarrow{*} ayb = x$.

Claim 2: Induction Step (Case 2: Stack empty in between)

Case 2: Stack becomes empty at state r before the end ($x = yz$).



- ▶ Both subcomputations $p \rightarrow r$ (on y) and $r \rightarrow q$ (on z) take $\leq k$ steps.
- ▶ By Induction Hypothesis: $A_{pr} \xRightarrow{*} y$ and $A_{rq} \xRightarrow{*} z$.
- ▶ Rule $A_{pq} \rightarrow A_{pr}A_{rq} \in G \implies A_{pq} \Rightarrow A_{pr}A_{rq} \xRightarrow{*} yz = x$.
- ▶ This completes the induction step and the entire proof!

Problem 1: Convert CFG to PDA

Practice Problem 1

Given the following Context-Free Grammar (CFG) G :

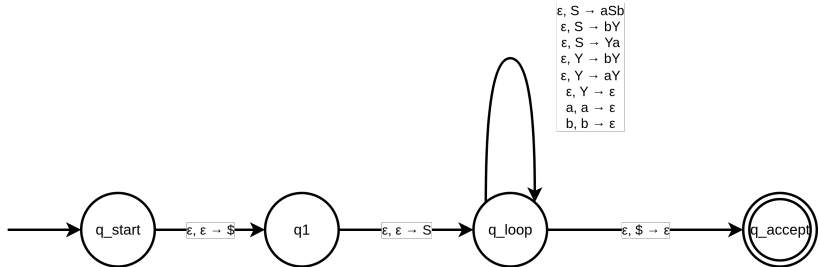
$$S \rightarrow aSb \mid bY \mid Ya$$

$$Y \rightarrow bY \mid aY \mid \varepsilon$$

Task:

- Construct an equivalent Pushdown Automaton (PDA) P that accepts the language generated by G .

Problem 1: Solution



Problem 2: PDA Construction

Practice Problem 2

Let $\Sigma = \{0, 1\}$ and let B be the collection of strings that contain at least one 1 in their second half. In other words:

$$B = \{uv \mid u \in \Sigma^*, v \in \Sigma^*1\Sigma^* \text{ and } |u| \geq |v|\}$$

Task:

- Give a Pushdown Automaton (PDA) that recognizes B .

Problem 2: Solution

